

1 Introduction

The Ukuqala Clinical Portal is a management system designed to help healthcare practitioners track patient information and vital signs digitally. In many clinical settings, data is still managed using manual paperwork, which leads to lost records and slow retrieval of patient history. This project aims to solve that by providing a fast, secure web interface where clinical data can be stored, edited, and deleted efficiently.

The system is built as a full-stack application. It allows doctors or nurses to log in securely, view a directory of patients, and manage clinical logs. The focus of the project was not just on making it work, but on making sure the code is professional, modular, and easy to deploy using Docker.

2 System Architecture

The project follows a standard Three-Tier Architecture. This structure ensures that the data, the logic, and the user interface are kept separate, making the system easier to maintain.

- **Frontend (React):** I used React with Vite for the user interface. The goal was a "human-centric" design, meaning it is clean and focuses on readability for a medical environment.
- **Backend (Node.js/Express):** The backend handles the "logic" of the app. It processes requests, checks user authentication, and communicates with the database.
- **Database (PostgreSQL):** All patient records and clinical logs are stored in a relational PostgreSQL database to ensure data integrity.

3 Implementation Details

A major part of the development was refactoring the code to meet high-quality standards. Originally, the frontend had a lot of "fat components"—where the UI code and the API logic were mixed together.

To fix this, I implemented a **Separation of Concerns**. I created a dedicated directory (`src/api`) to hold all the Axios fetching logic. This way, if the API URL changes or we need to update how data is fetched, we only have to change it in one place instead of editing every single page.

For deployment, I used **Docker Compose**. This containerizes the backend, frontend, and database so that anyone can run the project on their machine with a single command without worrying about installing dependencies manually.

4 Challenges and Solutions

During the development phase, I faced several technical challenges that required deep debugging.

4.1 The Cascading Render Bug

While building the Patients Page, I ran into a performance issue where the page would enter an infinite loop of re-rendering. This happened because I was calling `setLoading` and `setPatients` inside a `useEffect` hook. Because the function was being recreated on every render, it kept triggering itself.

Solution: I solved this by using the `useCallback` hook to stabilize the data-fetching function. By ensuring the function reference stayed the same unless its dependencies changed, the infinite loop was stopped, and the app became much faster.

4.2 Vite Transformation Errors

Another challenge occurred during the API refactoring. The Vite compiler started throwing "Unexpected Token" errors and refused to build the project. After investigating, I realized I had accidentally put JSX (HTML-like) code into a pure `.js` file.

Solution: I learned that Vite treats `.js` and `.jsx` files differently. I renamed my UI files to `.jsx` and kept my logic-only API files as `.js`. This fixed the transformation error immediately.

4.3 Docker Networking and Persistence

Initially, my backend container could not "talk" to my database container, and every time I stopped Docker, all my patient data was deleted.

Solution: I configured a custom network in the `docker-compose.yml` file so the services could find each other by name. I also added "Volumes" to the PostgreSQL service, which ensures the patient data stays saved on my hard drive even when the containers are turned off.

5 Conclusion

The Ukuqala Clinical Portal successfully meets the goals of providing a secure and manageable health record system. Through this project, I improved my skills in React state management, backend routing, and containerization. While the challenges like cascading renders were difficult to debug at first, solving them led to a much more stable and professional application. The system is now ready for deployment and serves as a strong base for future clinical features.